

Organization and Functionality of the Von Neumann Machine

Computer Architecture

SKU 2.2.1

Introduction

- ▶ • The Von Neumann architecture is a computer design model proposed by John Von Neumann in 1945.
- ▶ • It describes a system where the program instructions and data share the same memory space.
- ▶ • This model laid the foundation for modern computer systems.

Basic Components of the Von Neumann Machine

- ▶ 1. Central Processing Unit (CPU)
- ▶ 2. Memory Unit
- ▶ 3. Input Unit
- ▶ 4. Output Unit
- ▶ 5. Control Unit

Central Processing Unit (CPU)

- ▶ • The CPU is the brain of the computer.
- ▶ • It consists of the Arithmetic Logic Unit (ALU) and the Control Unit (CU).
- ▶ • Performs data processing and instruction execution.

Memory Unit

- ▶ • Stores both program instructions and data.
- ▶ • Accessed by the CPU during instruction execution.
- ▶ • Divided into primary and secondary memory.

Control Unit

- ▶ • Directs the operation of the processor.
- ▶ • Fetches instructions from memory and decodes them.
- ▶ • Controls data flow between CPU, memory, and I/O devices.

Arithmetic Logic Unit (ALU)

- ▶ • Performs arithmetic operations like addition and subtraction.
- ▶ • Performs logical operations like AND, OR, NOT.
- ▶ • Works under the control of the control unit.

System Bus

- ▶ • A communication pathway that connects all major components.
- ▶ • Three main types of buses:
 - ▶ - Data Bus
 - ▶ - Address Bus
 - ▶ - Control Bus

Instruction Cycle

- ▶ • The sequence of steps followed by the CPU to execute an instruction:
 - ▶ 1. Fetch - Get instruction from memory.
 - ▶ 2. Decode - Determine the operation.
 - ▶ 3. Execute - Perform the operation.
 - ▶ 4. Store - Write result back to memory.

Advantages and Limitations

► Advantages:

- • Simplicity of design and implementation.
- • Cost-effective and widely used.

► Limitations:

- • Von Neumann bottleneck due to shared memory.
- • Sequential execution slows down performance.

Modern Applications

- ▶ • Basis for modern computers and microcontrollers.
- ▶ • Used in embedded systems, desktops, and servers.
- ▶ • Concepts extended in modern architectures like Harvard and RISC.

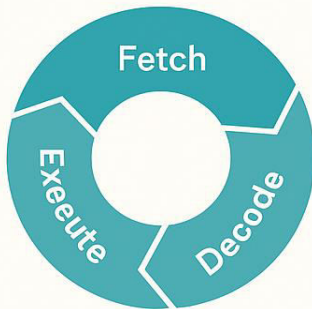
Summary

- ▶ • The Von Neumann machine integrates program and data in the same memory.
- ▶ • Consists of CPU, memory, I/O, and system bus.
- ▶ • Despite limitations, it remains a foundational model in computer architecture.

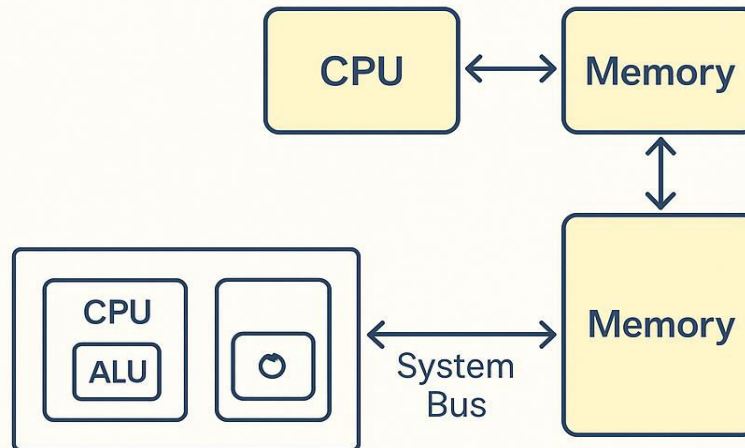
Diagram

The Organization and Functionality of the Von Neumann Machine

Computer Architecture - Undergraduate Level



Instruction Cycle



Instruction Level Parallelism (ILP)

Computer Architecture -
SKU 2.2.2

Introduction to Instruction Level Parallelism

- ▶ • Instruction Level Parallelism (ILP) refers to the ability of a processor to execute multiple instructions simultaneously.
- ▶ • ILP exploits parallel execution within a single program.
- ▶ • The goal is to improve performance without increasing clock frequency.
- ▶ • Modern CPUs achieve ILP through techniques like pipelining, superscalar execution, and out-of-order processing.

Types of ILP

- ▶ 1. ****Static ILP**** - Extracted at compile time by the compiler.
 - ▶ - Example: Loop unrolling.
- ▶ 2. ****Dynamic ILP**** - Extracted at runtime by the processor.
 - ▶ - Example: Out-of-order execution.
- ▶ • Both approaches aim to increase instruction throughput.

Pipelining

- ▶ • Divides instruction execution into stages: Fetch, Decode, Execute, Memory, Write-back.
- ▶ • Multiple instructions overlap in execution.
- ▶ • Increases CPU throughput but not single-instruction latency.
- ▶ • Common in RISC architectures.

Hazards Limiting ILP

- ▶ • ****Data Hazards**** - When instructions depend on the results of previous ones.
- ▶ • ****Control Hazards**** - Caused by branches and jumps.
- ▶ • ****Structural Hazards**** - Occur when hardware resources are insufficient.
- ▶ • Techniques like forwarding, branch prediction, and reservation stations mitigate hazards.

Superscalar Architecture

- ▶ • Executes more than one instruction per clock cycle.
- ▶ • Uses multiple execution units.
- ▶ • Employs dynamic scheduling and instruction issue logic.
- ▶ • Examples: Intel Pentium, AMD Ryzen.

Out-of-Order Execution

- ▶ • Allows instructions to execute as soon as operands are ready.
- ▶ • Avoids idle CPU cycles due to dependencies.
- ▶ • Uses structures like reorder buffers and reservation stations.
- ▶ • Improves overall CPU utilization and performance.

Speculative Execution

- ▶ • Processor guesses the outcome of branches and executes instructions ahead.
- ▶ • If prediction is correct, performance improves.
- ▶ • Incorrect predictions are rolled back.
- ▶ • Common in modern high-performance CPUs.

Techniques to Enhance ILP

- ▶ • Loop unrolling
- ▶ • Software pipelining
- ▶ • Dynamic scheduling
- ▶ • Register renaming
- ▶ • Branch prediction
- ▶ • Speculative execution

Advantages and Limitations of ILP

► ****Advantages:****

- • Increased performance.
- • Better CPU utilization.
- • Reduced execution time.

► ****Limitations:****

- • Complexity in hardware design.
- • Power and heat increase.
- • Diminishing returns beyond certain parallelism levels.

Conclusion

- ▶ • ILP is fundamental in modern processor design.
- ▶ • It enables faster execution without increasing clock frequency.
- ▶ • Advanced techniques like out-of-order execution and speculation are key.
- ▶ • Balancing performance and complexity remains a major challenge.

The Representation of Instructions at the Machine and Assembly Levels

Computer Architecture -
SKU 2.2.3

Overview

- ▶ • Understanding how instructions are represented in machine and assembly language
- ▶ • Relationship between high-level language and low-level representation
- ▶ • Instruction formats, addressing modes, and examples

Machine and Assembly Language

- ▶ • Machine Language: Binary representation directly executed by CPU
- ▶ • Assembly Language: Human-readable symbolic form of machine instructions
- ▶ • Each assembly instruction corresponds to one machine instruction

Instruction Format

- ▶ • Instructions are composed of fields:
- ▶ - Opcode: Specifies operation (e.g., ADD, LOAD)
- ▶ - Operands: Data or addresses
- ▶ - Addressing mode bits
- ▶ • Example (32-bit instruction): [Opcode | Address | Register | Mode]



Addressing Modes

- ▶ • Immediate: Operand is part of instruction
- ▶ • Direct: Address field contains operand address
- ▶ • Indirect: Address field points to address containing operand
- ▶ • Register: Operand in CPU register
- ▶ • Indexed: Effective address = base + index



Assembly Example

- ▶ Assembly Code:
 - ▶ `MOV R1, #5`
 - ▶ `ADD R2, R1, R3`
- ▶ Machine Code (Binary):
 - ▶ `1101 0001 0000 0101`
 - ▶ `0001 0010 0011 0000`

Instruction Cycle

- ▶ • Fetch: Retrieve instruction from memory
- ▶ • Decode: Interpret opcode and operands
- ▶ • Execute: Perform operation
- ▶ • Store: Write results back to memory or registers

Fetch

Decode

Execute

Store

Advantages of Assembly Language

- ▶ • Closer control over hardware
- ▶ • Efficient use of resources
- ▶ • Essential for performance-critical or embedded systems

Summary

- ▶ • Instructions bridge software and hardware
- ▶ • Machine level = binary form, executed by CPU
- ▶ • Assembly level = symbolic, readable form
- ▶ • Understanding both helps optimize and debug systems

The Concepts of Interrupts and I/O Operations

Computer Architecture -
SKU 2.2.4

Agenda & Learning Objectives

► Topics Covered:

- - The Input/Output (I/O) Problem
- - Programmed I/O
- - Interrupt-Driven I/O
- - Interrupt Handling
- - Types of Interrupts
- - DMA (Direct Memory Access)
- - Comparison of Techniques

► Learning Objectives:

- - Understand I/O challenges.
- - Explain Programmed I/O.
- - Define interrupts and their lifecycle.
- - Differentiate interrupt types.
- - Explain DMA.

The I/O Problem: A Mismatch in Speed

- ▶ CPU: ~1-10 Billion ops/sec
 - ▶ Memory: ~10-100 Million ops/sec
 - ▶ SSD: ~1-5 Million I/O ops
 - ▶ HDD: ~100-200 I/O ops
-
- ▶ Problem: CPU is much faster than I/O devices.
 - ▶ If CPU waits for I/O, performance drops.
 - ▶ Goal: Allow CPU to do useful work while waiting for I/O.

Technique 1: Programmed I/O (Polling)

- ▶ How it Works:
 - ▶ - CPU instructs device and polls status repeatedly.
 - ▶ - Transfers data when device is ready.
- ▶ Drawbacks:
 - ▶ - CPU wastes time checking device (busy-wait loop).
- ▶ Analogy: Checking for a delivery every 30 seconds.

Technique 2: Interrupt-Driven I/O

- ▶ How it Works:
 - ▶ - CPU issues command and continues other work.
 - ▶ - Device sends interrupt when ready.
 - ▶ - CPU pauses, services interrupt, resumes work.
- ▶ Analogy: Delivery company calls you when package arrives.

The Interrupt Handling Process

- ▶ Steps:
- ▶ 1. Finish current instruction
- ▶ 2. Save context
- ▶ 3. Identify interrupt source
- ▶ 4. Execute ISR (Interrupt Service Routine)
- ▶ 5. Restore context
- ▶ 6. Resume execution

Types of Interrupts

- ▶ 1. Hardware Interrupts - External devices (keyboard, timer)
- ▶ 2. Software Interrupts - System calls, exceptions
- ▶ 3. Internal Interrupts - Hardware errors or internal CPU events

Interrupt Vector Table (IVT) & Priorities

- ▶ IVT: A table in memory with ISR addresses.
- ▶ Example:
 - ▶ 0 → 0x0000...
 - ▶ 1 → 0x0004...
 - ▶ 2 → 0x0008...
- ▶ Priority:
 - ▶ - High-priority interrupts handled first.
 - ▶ - Low-priority can be masked.

Technique 3: Direct Memory Access (DMA)

- ▶ Problem: Interrupts inefficient for large transfers.
- ▶ DMA Solution:
 - ▶ - CPU sets up DMA controller (source, dest, size).
 - ▶ - DMA handles transfer directly between device & memory.
 - ▶ - CPU interrupted only after completion.

Comparison of I/O Techniques

- ▶ Programmed I/O: High CPU use, low efficiency.
- ▶ Interrupt-Driven I/O: Moderate CPU use, better efficiency.
- ▶ DMA: Minimal CPU use, very high efficiency.
- ▶ Modern systems combine all techniques.

Summary

- ▶ - I/O devices are slower than CPUs.
- ▶ - Programmed I/O wastes CPU cycles.
- ▶ - Interrupts improve efficiency.
- ▶ - DMA allows high-speed data transfer without CPU load.

Assembly Language Program Segments with Subroutine Calls

Computer Architecture

SKU 2.2.5

Introduction

- ▶ Assembly language: Low-level programming language closely related to machine language.
- ▶ Program segments: Divide program into logical blocks.
- ▶ Subroutine: Sequence of instructions that performs a specific task.

Structure of Assembly Language Programs

- ▶ Main Program: Controls overall execution.
- ▶ Subroutine(s): Independent program segments.
- ▶ Data Segment: Stores variables and constants.
- ▶ Stack Segment: For function calls and returns.

What is a Subroutine?

- ▶ Definition: A subroutine is a reusable code block.
- ▶ Advantages:
 - ▶ - Code reuse
 - ▶ - Modular programming
 - ▶ - Easier debugging
- ▶ Types:
 - ▶ - Procedure / Function
 - ▶ - Macro

Calling a Subroutine

- ▶ Instructions:
- ▶ CALL - Transfers control to subroutine
- ▶ RET - Returns control to main program
- ▶ Stack - Stores return address

Example: Simple Assembly Program

- ▶ ; Main Program
- ▶ MOV AX, 5
- ▶ CALL DOUBLE
- ▶ MOV BX, AX

- ▶ ; Subroutine
- ▶ DOUBLE:
- ▶ ADD AX, AX
- ▶ RET

- ▶ Explanation:
- ▶ Main program moves value into AX, calls subroutine DOUBLE.
- ▶ DOUBLE doubles AX value and returns.

Stack Operations in Subroutine Calls

- ▶ Steps:
- ▶ 1. Push return address onto stack.
- ▶ 2. Jump to subroutine.
- ▶ 3. Execute subroutine instructions.
- ▶ 4. RET pops return address.

Advantages of Using Subroutines

- ▶ Reduces code duplication.
- ▶ Simplifies program maintenance.
- ▶ Supports structured programming.
- ▶ Efficient memory usage.

Summary

- ▶ Assembly programs are divided into segments.
- ▶ Subroutines make code modular and reusable.
- ▶ CALL and RET are essential for subroutine handling.
- ▶ Stack is key for return addresses and local data.

High-Level Programs Are Implemented at the Machine Language Level

Computer Architecture -
SKU 2.2.6

Introduction

- ▶ High-level languages (C, Java, Python) are human-readable.
- ▶ CPU executes only machine code (binary).
- ▶ Translation Stages:
 - ▶ - Compilation
 - ▶ - Assembly
 - ▶ - Linking
 - ▶ - Execution

Levels of Program Representation

- ▶ High-Level: $a = b + c$; (Human-readable)
- ▶ Assembly: MOV AX, B / ADD AX, C / MOV A, AX (Symbolic)
- ▶ Machine: 10110000 00000001 ... (Binary)

The Role of the Compiler

- ▶ Compiler converts high-level code into assembly or object code.
- ▶ Main Tasks:
 - ▶ - Syntax Analysis
 - ▶ - Code Optimization
 - ▶ - Code Generation
- ▶ Output: Assembly or Machine-Level Instructions.

Assembler and Linker

- ▶ Assembler: Translates assembly → machine code (object file).
- ▶ Linker: Combines object files and libraries → executable.
- ▶ Loader: Loads executable into memory for execution.
- ▶ Flow: Source → Compiler → Assembler → Linker → Loader → CPU

Machine Language Execution

- ▶ Machine language: binary instructions (0s & 1s).
- ▶ Each instruction has: Opcode + Operands.
- ▶ Example (x86):
 - ▶ MOV AX, 0005h → Binary: B8 05 00
- ▶ CPU performs Fetch-Decode-Execute cycle.

Example: From C Code to Machine Code

- ▶ High-Level Code:

- ▶ `int sum(int a, int b) { return a + b; }`

- ▶ Assembly:

- ▶ `MOV AX, [a]`

- ▶ `ADD AX, [b]`

- ▶ `RET`

- ▶ Machine Code:

- ▶ `8B 45 08`

- ▶ `03 45 0C`

- ▶ `C3`

The Translation Hierarchy

- ▶ Source Code (C, Java): Written by programmer
- ▶ ↓
- ▶ Compiler Output (Assembly): Symbolic low-level code
- ▶ ↓
- ▶ Assembler Output (Object Code): Binary format
- ▶ ↓
- ▶ Executable Program: Run by CPU

Machine Organization View

- ▶ Instructions & data reside in main memory.
- ▶ CPU executes using:
 - ▶ - Control Unit (fetch/decode)
 - ▶ - ALU (execute)
 - ▶ - Registers (temporary storage)
- ▶ High-level code maps to hardware operations.

Importance of Understanding Machine Level

- ▶ Benefits:
- ▶ - Efficient programming & debugging
- ▶ - Basis for compiler, OS, and embedded design
- ▶ - Improves understanding of:
 - ▶ * Instruction cycles
 - ▶ * Performance optimization
 - ▶ * Low-level control

Summary

- ▶ High-level programs → machine language before execution.
- ▶ Steps: Compilation → Assembly → Linking → Loading.
- ▶ Machine instructions executed by CPU.
- ▶ Bridges software & hardware understanding.